LAB ASSIGNMNET 12.3

Subject : AI Assistant Coding

Hall Ticket No : 2303A52465

Name : M.Avinash

Batch : 50

Date : 13/3/26

Task-1 :

Prompt : Analyse the given scenario and generate a program that stores student records(Name, Roll Number, CGPA), using sorting Algorithms Merge and Quick Sort. Measure and compare runtime performance for large datasets and Write a function to display the top 10 students based on CGPA.

```
# SR University's Training and Placement Cell needs to shortlist candidates
# efficiently during campus placements. Student records must be sorted by CGPA in descending order.
# Analyse the given scenario and generate a program that stores student records(Name, Roll Number, CGPA), using sorting Algorithms Merge and Quick Sor
# Measure and compare runtime performance for large datasets and Write a function to display the top 10 students based on CGPA.
import (class) Student
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} (Roll No: {self.roll_number}, CGPA: {self.cgpa})"
def merge_sort(students):
    if len(students) > 1:
        mid = len(students) // 2
        left_half = students[:mid]
        right_half = students[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        while i < len(left_half) and j < len(right_half):
            if left_half[i].cgpa > right_half[j].cgpa:
                students[k] = left_half[i]
                i += 1
            else:
                students[k] = right_half[j]
                j += 1
            k += 1

        while i < len(left_half):
            students[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            students[k] = right_half[j]
            j += 1
            k += 1
def quick_sort(students):
    if len(students) <= 1:
        return students
    else:
        pivot = students[0]
        left = [x for x in students[1:] if x.cgpa > pivot.cgpa]
        right = [x for x in students[1:] if x.cgpa <= pivot.cgpa]
        return quick_sort(left) + [pivot] + quick_sort(right)
def display_top_students(students, n=10):
    print(f"Top {n} Students based on CGPA:")
    for student in students[:n]:
        print(student)
```

```python
# Sample data for testing
students = [
    Student("Alice", "001", 3.8),
    Student("Bob", "002", 3.6),
    Student("Charlie", "003", 3.9),
    Student("David", "004", 3.7),
    Student("Eve", "005", 3.5),
    Student("Frank", "006", 3.4),
    Student("Grace", "007", 3.9),
    Student("Heidi", "008", 3.2),
    Student("Ivan", "009", 3.1),
    Student("Judy", "010", 3.0)
]
# Measure runtime for Merge Sort
start_time = time.time()
merge_sort(students)
end_time = time.time()
print(f"Merge Sort Runtime: {end_time - start_time:.6f} seconds")
display_top_students(students)
# Measure runtime for Quick Sort
start_time = time.time()
sorted_students = quick_sort(students)
end_time = time.time()
print(f"Quick Sort Runtime: {end_time - start_time:.6f} seconds")
display_top_students(sorted_students)
```

Output :

```
Merge Sort Runtime: 0.000021 seconds
Quick Sort Runtime: 0.000020 seconds
Top 10 Students based on CGPA:
Grace (Roll No: 007, CGPA: 3.9)
Charlie (Roll No: 003, CGPA: 3.9)
Alice (Roll No: 001, CGPA: 3.8)
David (Roll No: 004, CGPA: 3.7)
Bob (Roll No: 002, CGPA: 3.6)
Eve (Roll No: 005, CGPA: 3.5)
Frank (Roll No: 006, CGPA: 3.4)
Heidi (Roll No: 008, CGPA: 3.2)
Ivan (Roll No: 009, CGPA: 3.1)
Judy (Roll No: 010, CGPA: 3.0)
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12>
```

Task-2 :

Prompt : Write a Python implementation of Bubble Sort and use AI to generate inline comments explaining key logic (like swapping, passes, and termination) and provide time complexity analysis.

```python
# Write a Python implementation of Bubble Sort
# and use AI to generate inline comments explaining key logic (like swapping, passes, and termination) and provide time complexity analysis.
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  # Swap
                swapped = True  # Set flag to True if a swap occurred
        # If no swapping occurred, the array is already sorted, so we can terminate early
        if not swapped:
            break
# Time complexity analysis:
# The worst-case and average-case time complexity of Bubble Sort is O(n^2) because it requires two nested loops to compare each element with every other element.
#  The best-case time complexity is O(n) when the array is already sorted, as it only requires one pass to verify that the array is in order.
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("Original array:", arr)
bubble_sort(arr)
print("Sorted array:", arr)
```

Output :

```
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12> & C:/
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12>
```

Task-3 :

Prompt : Generate implementation for Quick Sort and Merge Sort using recursion.
and Use AI to provide with partially completed functions for recursion, complete the missing logic and add docstrings, and Compare both algorithms on random, sorted, and reverse-sorted lists.

```python
# Generate implementation for Quick Sort and Merge Sort using recursion.
# and Use AI to provide with partially completed functions for recursion, complete the missing logic and add docstrings.
# and Compare both algorithms on random, sorted, and reverse-sorted lists.
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) > 1:
        mid = len(arr) // 2  # Finding the mid of the array
        left_half = arr[:mid]  # Dividing the array elements into 2 halves
        right_half = arr[mid:]

        merge_sort(left_half)  # Sorting the first half
        merge_sort(right_half)  # Sorting the second half

        i = j = k = 0

        # Copy data to temp arrays L[] and R[]
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Checking if any element was left
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
```

```python
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]  # Choosing the first element as the pivot
        left = [x for x in arr[1:] if x < pivot]  # Elements less than the pivot
        right = [x for x in arr[1:] if x >= pivot]  # Elements greater than or equal to the pivot
        return quick_sort(left) + [pivot] + quick_sort(right)  # Recursively sort and combine
# Example usage
import random
# Generate random, sorted, and reverse-sorted lists
random_list = [random.randint(1, 100) for _ in range(10)]
sorted_list = sorted(random_list)
reverse_sorted_list = sorted(random_list, reverse=True)
print("Random List:", random_list)
print("Sorted List:", sorted_list)
print("Reverse Sorted List:", reverse_sorted_list)
# Compare Merge Sort
merge_sort(random_list.copy())
print("Merge Sorted Random List:", random_list)
merge_sort(sorted_list.copy())
print("Merge Sorted Sorted List:", sorted_list)
merge_sort(reverse_sorted_list.copy())
print("Merge Sorted Reverse Sorted List:", reverse_sorted_list)
# Compare Quick Sort
print("Quick Sorted Random List:", quick_sort(random_list.copy()))
print("Quick Sorted Sorted List:", quick_sort(sorted_list.copy()))
print("Quick Sorted Reverse Sorted List:", quick_sort(reverse_sorted_list.copy()))
# Best, Average, and Worst-case time complexity for both algorithms:
# Merge Sort:
# - Best-case: O(n log n) (occurs when the array is already sorted)
# - Average-case: O(n log n)
# - Worst-case: O(n log n) (occurs when the array is reverse sorted)
# Quick Sort:
# - Best-case: O(n log n) (occurs when the pivot divides the array into two equal halves)
# - Average-case: O(n log n)
# - Worst-case: O(n^2) (occurs when the smallest or largest element is always chosen as the pivot, such as in a sorted or reverse sorted array)
```

Output :

```
Random List: [28, 8, 70, 74, 95, 92, 14, 31, 20, 3]
Sorted List: [3, 8, 14, 20, 28, 31, 70, 74, 92, 95]
Reverse Sorted List: [95, 92, 74, 70, 31, 28, 20, 14, 8, 3]
Merge Sorted Random List: [28, 8, 70, 74, 95, 92, 14, 31, 20, 3]
Merge Sorted Sorted List: [3, 8, 14, 20, 28, 31, 70, 74, 92, 95]
Merge Sorted Reverse Sorted List: [95, 92, 74, 70, 31, 28, 20, 14, 8, 3]
Quick Sorted Random List: [3, 8, 14, 20, 28, 31, 70, 74, 92, 95]
Quick Sorted Sorted List: [3, 8, 14, 20, 28, 31, 70, 74, 92, 95]
Quick Sorted Reverse Sorted List: [3, 8, 14, 20, 28, 31, 70, 74, 92, 95]
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12>
```

Task-4 :

Prompt : Analyse the given scenario and Use AI to generate the most efficient search and sort algorithms for this use case.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

```python
# Analyse the given scenario and Use AI to generate the most efficient search and sort algorithms for this use case.
# Expected Output:
# • A table mapping operation → recommended algorithm → justification.
# • Working Python functions for searching and sorting the inventory.
def search_product_by_id(inventory, product_id):
    """
    Searches for a product in the inventory by its ID.

    Parameters:
    inventory (list): A list of product dictionaries.
    product_id (str): The ID of the product to search for.

    Returns:
    dict: The product dictionary if found, else None.
    """
    # Using a dictionary for O(1) average time complexity search
    product_dict = {product['id']: product for product in inventory}
    return product_dict.get(product_id, None)
def search_product_by_name(inventory, product_name):
    """
    Searches for a product in the inventory by its name.
    Parameters:
    inventory (list): A list of product dictionaries.
    product_name (str): The name of the product to search for.
    Returns:
    dict: The product dictionary if found, else None.
    """
    # Using a dictionary for O(1) average time complexity search
    product_dict = {product['name']: product for product in inventory}
    return product_dict.get(product_name, None)
def sort_products_by_price(inventory):
    """
    Sorts the inventory products by price in ascending order.
    Parameters:
    inventory (list): A list of product dictionaries.
    Returns:
    list: A new list of products sorted by price.
    """
    # Using Timsort (Python's built-in sort) which has O(n log n) time complexity
    return sorted(inventory, key=lambda x: x['price'])
def sort_products_by_quantity(inventory):
    """
    Sorts the inventory products by stock quantity in ascending order.
    Parameters:
    inventory (list): A list of product dictionaries.
    Returns:
    list: A new list of products sorted by stock quantity.
    """
    # Using Timsort (Python's built-in sort) which has O(n log n) time complexity
    return sorted(inventory, key=lambda x: x['quantity'])
```

```python
# Example inventory data
inventory = [
    {'id': 'P001', 'name': 'Product A', 'price': 10.99, 'quantity': 100},
    {'id': 'P002', 'name': 'Product B', 'price': 5.49, 'quantity': 200},
    {'id': 'P003', 'name': 'Product C', 'price': 20.00, 'quantity': 50},
    {'id': 'P004', 'name': 'Product D', 'price': 15.75, 'quantity': 150},
]
# Testing the functions
print("Search by ID (P002):", search_product_by_id(inventory, 'P002'))
print("Search by Name (Product C):", search_product_by_name(inventory, 'Product C'))
print("Products sorted by Price:", sort_products_by_price(inventory))
print("Products sorted by Quantity:", sort_products_by_quantity(inventory))
# Operation → Recommended Algorithm → Justification
# 1. Search by ID → Hash Table (Dictionary) → Provides O(1) average time complexity for lookups, making it efficient for searching by unique identifiers.
# 2. Search by Name → Hash Table (Dictionary) → Similar to searching by ID
# 3. Sort by Price → Timsort (Python's built-in sort) → Timsort is optimized for real-world data and has O(n log n) time complexity, making it efficient for sorting large datasets.
# 4. Sort by Quantity → Timsort (Python's built-in sort) → Same justification as sorting by price, as it is efficient for sorting large datasets based on any key.
```

Output :

```
Search by ID (P002): {'id': 'P002', 'name': 'Product B', 'price': 5.49, 'quantity': 200}
Search by Name (Product C): {'id': 'P003', 'name': 'Product C', 'price': 20.0, 'quantity': 50}
Products sorted by Price: [{'id': 'P002', 'name': 'Product B', 'price': 5.49, 'quantity': 200}, {'id': 'P001', 'name'
e': 15.75, 'quantity': 150}, {'id': 'P003', 'name': 'Product C', 'price': 20.0, 'quantity': 50}]
Products sorted by Quantity: [{'id': 'P003', 'name': 'Product C', 'price': 20.0, 'quantity': 50}, {'id': 'P001', 'nam
ice': 15.75, 'quantity': 150}, {'id': 'P002', 'name': 'Product B', 'price': 5.49, 'quantity': 200}]
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12>
```

Task-5 :

Prompt : Analyse the given scenario and use AI to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price). Implement sorting algorithms to rank stocks by percentage change and a search function that retrieves stock data instantly when a stock symbol is entered. Optimize sorting with Heap Sort and searching with Hash Maps and Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

0

```python
# Analyse the given scenario and use AI to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
# Implement sorting algorithms to rank stocks by percentage change and a search function that retrieves stock data instantly when a stock symbol is entered.
# Optimize sorting with Heap Sort and searching with Hash Maps
# and Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.
import random
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price

    def percentage_change(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100

    def __repr__(self):
        return f"{self.symbol} (Opening: {self.opening_price}, Closing: {self.closing_price}, Change: {self.percentage_change():.2f}%)"
def heapify(arr, n, i):
    largest = i  # Initialize largest as root
    l = 2 * i + 1  # left = 2*i + 1
    r = 2 * i + 2  # right = 2*i + 2

    # If left child is larger than root
    if l < n and arr[l].percentage_change() > arr[largest].percentage_change():
        largest = l

    # If right child is larger than largest so far
    if r < n and arr[r].percentage_change() > arr[largest].percentage_change():
        largest = r

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # Swap

        # Recursively heapify the affected sub-tree
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)

    # Build a maxheap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # Swap
        heapify(arr, i, 0)
```

```python
def search_stock_by_symbol(stock_map, symbol):
    """
    Searches for a stock in the stock map by its symbol.

    Parameters:
    stock_map (dict): A dictionary mapping stock symbols to Stock objects.
    symbol (str): The symbol of the stock to search for.

    Returns:
    Stock: The Stock object if found, else None.
    """
    return stock_map.get(symbol, None)
# Simulate stock price data
stock_symbols = ['AAPL', 'GOOGL', 'MSFT', 'AMZN', 'TSLA']
stocks = []
for symbol in stock_symbols:
    opening_price = random.uniform(100, 500)
    closing_price = opening_price + random.uniform(-20, 20)  # Simulate gain/loss
    stocks.append(Stock(symbol, opening_price, closing_price))
# Create a hash map for efficient searching
stock_map = {stock.symbol: stock for stock in stocks}
# Sort stocks by percentage change using Heap Sort
heap_sort(stocks)
print("Stocks sorted by percentage change:")
for stock in stocks:
    print(stock)
# Search for a specific stock symbol
search_symbol = 'AAPL'
found_stock = search_stock_by_symbol(stock_map, search_symbol)
if found_stock:
    print(f"\nStock found for symbol '{search_symbol}': {found_stock}")
else:
    print(f"\nStock with symbol '{search_symbol}' not found.")

# Performance Comparison:
# Sorting with Heap Sort
# - Time Complexity: O(n log n) in all cases (best, average, worst)
# - Space Complexity: O(1) (in-place sorting)
# Sorting with sorted() (Timsort)
# - Time Complexity: O(n log n) in all cases (best, average, worst)
# - Space Complexity: O(n) (not in-place)
# Searching with Hash Maps (Dictionaries)
# - Time Complexity: O(1) average case, O(n) worst case (due to collisions)
# - Space Complexity: O(n) (for storing the hash map)
```

Output :

```
Stocks sorted by percentage change:
AAPL (Opening: 138.4893666275976, Closing: 125.8676554712701, Change: -9.11%)
GOOGL (Opening: 165.83591084653403, Closing: 159.1238921500739, Change: -4.05%)
AMZN (Opening: 358.1405430158354, Closing: 352.37833337491844, Change: -1.61%)
MSFT (Opening: 335.30192820377, Closing: 331.64172070085885, Change: -1.09%)
TSLA (Opening: 343.1217787322477, Closing: 346.43469259818545, Change: 0.97%)

Stock found for symbol 'AAPL': AAPL (Opening: 138.4893666275976, Closing: 125.8676554712701, Change: -9.11%)
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-12>
```